LeetCode Hot100 - 哈希表专题

---

1. 两数之和 【简单】

思路

核心思想：哈希表存储已遍历元素，O(1) 查找配对值

- 遍历数组，对于每个数 `num`，计算其配对值 `target - num`
- 用哈希表记录 `{数值: 索引}`，边遍历边查找
- 若配对值已在哈希表中，直接返回两个索引

时间复杂度：O(n)，空间复杂度：O(n)

代码

```python
class Solution:
    def twoSum(self, nums: List[int], target: int) -> List[int]:
        # 哈希表：数值 -> 索引
        hash_map = {}

        for i, num in enumerate(nums):
            complement = target - num  # 计算配对值

            # 如果配对值已存在，返回结果
            if complement in hash_map:
                return [hash_map[complement], i]

            # 将当前数存入哈希表
            hash_map[num] = i

        return []
```

---

49. 字母异位词分组 【中等】

思路

核心思想：异位词排序后相同，以排序结果作为哈希表的 key

- 字母异位词的特点：字符组成相同，顺序不同
- 对每个字符串排序，排序后的结果作为分组依据
- 用哈希表 `{排序后字符串: [原字符串列表]}` 进行分组

时间复杂度：O(n * k * log k)，其中 k 为字符串最大长度

代码

```python
class Solution:
    def groupAnagrams(self, strs: List[str]) -> List[List[str]]:
        from collections import defaultdict
```

```
        # 哈希表：排序后的字符串 -> 原字符串列表
        groups = defaultdict(list)

        for s in strs:
            # 排序字符串作为 key（tuple 可哈希）
            key = tuple(sorted(s))
            groups[key].append(s)

        return list(groups.values())
```

替代方案：用字符计数作为 key（避免排序）

```
class Solution:
    def groupAnagrams(self, strs: List[str]) -> List[List[str]]:
        from collections import defaultdict

        groups = defaultdict(list)

        for s in strs:
            # 统计26个字母出现次数作为 key
            count = [0] * 26
            for c in s:
                count[ord(c) - ord('a')] += 1
            groups[tuple(count)].append(s)

        return list(groups.values())
```

---

128. 最长连续序列 【中等】

思路

核心思想：哈希集合 + 只从序列起点开始计数

- 将数组存入哈希集合，实现 O(1) 查找
- 遍历每个数，**只有当 `num - 1` 不存在时**，才开始计数（确保从序列起点开始）
- 从起点向后连续查找 `num + 1, num + 2, ...`，统计长度

关键优化：跳过非起点，避免重复计算，保证 O(n) 复杂度

时间复杂度：O(n)，空间复杂度：O(n)

代码

```
class Solution:
    def longestConsecutive(self, nums: List[int]) -> int:
        if not nums:
            return 0

        # 转为集合，O(1) 查找
        num_set = set(nums)
        max_length = 0
```

```
        for num in num_set:
            # 关键：只从序列起点开始计数（前一个数不存在）
            if num - 1 not in num_set:
                current_num = num
                current_length = 1

                # 向后连续查找
                while current_num + 1 in num_set:
                    current_num += 1
                    current_length += 1

                max_length = max(max_length, current_length)

        return max_length
```

哈希表专题总结

题目	核心技巧	哈希表用途
两数之和	边遍历边查找	存储 {值: 索引}
字母异位词分组	排序/计数作为 key	分组聚合
最长连续序列	只从起点计数	O(1) 存在性判断

哈希表核心价值：将 O(n) 查找优化为 O(1)

LeetCode Hot100 - 双指针专题

283. 移动零 【简单】

思路

快慢指针原地交换

- `slow` 指向下一个非零元素应放置的位置
- `fast` 遍历数组，遇到非零元素就与 `slow` 位置交换
- 核心：保持 `[0, slow)` 区间全是非零元素

代码

```
class Solution:
    def moveZeroes(self, nums: List[int]) -> None:
        slow = 0  # 指向下一个非零元素应该放置的位置
        for fast in range(len(nums)):
            if nums[fast] != 0:
                # 交换，将非零元素移到前面
                nums[slow], nums[fast] = nums[fast], nums[slow]
                slow += 1
```

---

11. 盛最多水的容器 【中等】

思路

左右双指针 + 贪心

- 两指针从两端向中间移动
- 面积 = `min(height[l], height[r]) * (r - l)`
- **关键**：每次移动较短的那根柱子（移动长的不可能得到更大面积）

代码

```
class Solution:
    def maxArea(self, height: List[int]) -> int:
        l, r = 0, len(height) - 1
        ans = 0
        while l < r:
            # 计算当前面积，取较短边 * 宽度
            area = min(height[l], height[r]) * (r - l)
            ans = max(ans, area)
            # 移动较短的一边，才可能找到更大面积
            if height[l] < height[r]:
                l += 1
            else:
                r -= 1
        return ans
```

---

15. 三数之和 【中等】

思路

排序 + 固定一个数 + 双指针找两数之和

- 先排序，方便去重和双指针
- 固定 `nums[i]`，在 `[i+1, n-1]` 范围用双指针找 `target = -nums[i]`
- **去重**：跳过相同的 `i`、`l`、`r`

代码

```
class Solution:
    def threeSum(self, nums: List[int]) -> List[List[int]]:
        nums.sort()  # 排序是关键
        n = len(nums)
        ans = []

        for i in range(n - 2):
            # 去重：跳过相同的 nums[i]
            if i > 0 and nums[i] == nums[i - 1]:
                continue
            # 剪枝：最小三数之和已经大于0
            if nums[i] + nums[i + 1] + nums[i + 2] > 0:
```

```
            break
        # 剪枝：当前数加最大两数仍小于0
        if nums[i] + nums[n - 2] + nums[n - 1] < 0:
            continue

        # 双指针找两数之和等于 -nums[i]
        l, r = i + 1, n - 1
        target = -nums[i]
        while l < r:
            s = nums[l] + nums[r]
            if s == target:
                ans.append([nums[i], nums[l], nums[r]])
                l += 1
                r -= 1
                # 去重：跳过相同的 l 和 r
                while l < r and nums[l] == nums[l - 1]:
                    l += 1
                while l < r and nums[r] == nums[r + 1]:
                    r -= 1
            elif s < target:
                l += 1
            else:
                r -= 1

    return ans
```

---

42. 接雨水 【困难】

思路

双指针 + 动态维护左右最大高度

- 核心公式：位置 `i` 能接的雨水 = `min(左边最高, 右边最高) - height[i]`
- 用 `left_max` 和 `right_max` 分别记录左右两侧遍历过的最大高度
- **关键**：哪边最大值小，就处理哪边（因为接水量由较小的一边决定）

代码

```
class Solution:
    def trap(self, height: List[int]) -> int:
        if not height:
            return 0

        l, r = 0, len(height) - 1
        left_max = right_max = 0  # 记录左右两侧的最大高度
        ans = 0

        while l < r:
            left_max = max(left_max, height[l])
            right_max = max(right_max, height[r])
```

```
            # 哪边最大值小，就计算哪边（短板决定接水量）
            if left_max < right_max:
                # 左边是短板，计算左边能接的水
                ans += left_max - height[l]
                l += 1
            else:
                # 右边是短板，计算右边能接的水
                ans += right_max - height[r]
                r -= 1

        return ans
```

双指针技巧总结

题目	指针类型	核心思想
移动零	快慢指针	慢指针维护有效区间
盛水容器	左右指针	贪心移动短板
三数之和	固定+左右指针	排序+去重+双指针
接雨水	左右指针	动态维护两侧最值

LeetCode Hot100 - 滑动窗口专题

---

3. 无重复字符的最长子串 【中等】

思路

滑动窗口 + 哈希表

- 维护一个窗口 `[left, right]`，保证窗口内无重复字符
- 用哈希表记录每个字符最后出现的位置
- 遇到重复字符时，将 `left` 跳到重复字符的下一个位置
- 每次更新最大长度 `right - left + 1`

核心: 右指针扩展，左指针收缩跳跃

代码

```
class Solution:
    def lengthOfLongestSubstring(self, s: str) -> int:
        char_index = {}  # 记录字符最后出现的位置
        left = 0
        max_len = 0

        for right, char in enumerate(s):
            # 如果字符已存在且在窗口内，移动左边界
            if char in char_index and char_index[char] >= left:
```

```
                left = char_index[char] + 1

            char_index[char] = right  # 更新字符位置
            max_len = max(max_len, right - left + 1)

        return max_len
```

---

438. 找到字符串中所有字母异位词 【中等】

思路

固定窗口 + 字符计数

- 异位词 = 字符种类和个数相同
- 维护长度为 `len(p)` 的固定窗口
- 用数组/哈希表统计窗口内字符频率，与 p 的频率比较
- 窗口滑动：右边加一个字符，左边减一个字符

优化: 维护"有效字符数"，避免每次比较整个频率表

代码

```python
class Solution:
    def findAnagrams(self, s: str, p: str) -> List[int]:
        if len(s) < len(p):
            return []

        result = []
        p_count = [0] * 26
        s_count = [0] * 26

        # 统计 p 的字符频率
        for char in p:
            p_count[ord(char) - ord('a')] += 1

        # 滑动窗口
        for i in range(len(s)):
            # 右边界字符进入窗口
            s_count[ord(s[i]) - ord('a')] += 1

            # 左边界字符离开窗口（窗口大小超过 len(p)）
            if i >= len(p):
                s_count[ord(s[i - len(p)]) - ord('a')] -= 1

            # 比较频率是否相同
            if s_count == p_count:
                result.append(i - len(p) + 1)

        return result
```

---

560. 和为 K 的子数组 【中等】

思路

前缀和 + 哈希表

- **注意**: 此题不能用滑动窗口，因为数组包含负数
- 前缀和定义: `prefix[i] = nums[0] + nums[1] + ... + nums[i-1]`
- 子数组和 `nums[i..j] = prefix[j+1] - prefix[i]`
- 若 `prefix[j] - prefix[i] = k`，则 `prefix[i] = prefix[j] - k`
- 用哈希表记录每个前缀和出现的次数

核心: 找满足 `prefix[j] - k` 的前缀和个数

代码

```python
class Solution:
    def subarraySum(self, nums: List[int], k: int) -> int:
        count = 0
        prefix_sum = 0
        # 前缀和出现次数，初始化 {0: 1} 表示前缀和为0出现1次
        prefix_count = {0: 1}

        for num in nums:
            prefix_sum += num

            # 查找是否存在 prefix_sum - k
            if prefix_sum - k in prefix_count:
                count += prefix_count[prefix_sum - k]

            # 更新当前前缀和的出现次数
            prefix_count[prefix_sum] = prefix_count.get(prefix_sum, 0) + 1

        return count
```

---

239. 滑动窗口最大值 【困难】

思路

单调递减队列

- 维护一个**单调递减的双端队列**，存储元素下标
- 队首始终是当前窗口的最大值
- 新元素入队前，将队尾所有比它小的元素弹出（它们不可能成为最大值）
- 检查队首元素是否超出窗口范围，超出则弹出

核心: 队列中元素从队首到队尾单调递减，保证队首是最大值

代码

```python
from collections import deque
```

```python
class Solution:
    def maxSlidingWindow(self, nums: List[int], k: int) -> List[int]:
        result = []
        dq = deque()  # 存储下标，保持对应值单调递减

        for i in range(len(nums)):
            # 1. 移除队首超出窗口范围的元素
            if dq and dq[0] <= i - k:
                dq.popleft()

            # 2. 维护单调性：移除队尾所有比当前元素小的
            while dq and nums[dq[-1]] < nums[i]:
                dq.pop()

            # 3. 当前元素下标入队
            dq.append(i)

            # 4. 窗口形成后，记录最大值（队首）
            if i >= k - 1:
                result.append(nums[dq[0]])

        return result
```

---

76. 最小覆盖子串 【困难】

思路

滑动窗口 + 双指针

- 用两个哈希表: `need` 记录目标字符需求， `window` 记录窗口内字符
- 维护 `valid` 表示满足需求的字符种类数
- **扩展**: 右指针移动，将字符加入窗口
- **收缩**: 当 `valid == len(need)` 时，左指针收缩，尝试缩小窗口
- 收缩过程中更新最小覆盖子串

核心: 先扩展找到可行解，再收缩优化解

代码

```python
from collections import Counter

class Solution:
    def minWindow(self, s: str, t: str) -> str:
        if not t or not s:
            return ""

        need = Counter(t)  # 目标字符需求
        window = {}        # 窗口内字符计数

        left = 0
        valid = 0          # 满足需求的字符种类数
```

```python
    start = 0           # 最小覆盖子串起始位置
    min_len = float('inf')

    for right, char in enumerate(s):
        # 扩展窗口
        if char in need:
            window[char] = window.get(char, 0) + 1
            if window[char] == need[char]:
                valid += 1

        # 收缩窗口
        while valid == len(need):
            # 更新最小覆盖子串
            if right - left + 1 < min_len:
                min_len = right - left + 1
                start = left

            # 移出左边界字符
            left_char = s[left]
            left += 1
            if left_char in need:
                if window[left_char] == need[left_char]:
                    valid -= 1
                window[left_char] -= 1

    return "" if min_len == float('inf') else s[start:start + min_len]
```

滑动窗口模板总结

```python
def sliding_window(s):
    left = 0
    window = {}  # 或其他数据结构
    result = ...

    for right in range(len(s)):
        # 1. 扩展窗口，加入 s[right]
        ...

        # 2. 判断是否需要收缩
        while 需要收缩:
            # 3. 更新结果（如果在收缩时更新）
            ...
            # 4. 收缩窗口，移出 s[left]
            ...
            left += 1

        # 5. 更新结果（如果在扩展后更新）
        ...
```

```
    return result
```

关键点:

1. **何时扩展**: 每次迭代右指针都会扩展
2. **何时收缩**: 根据题目条件判断（窗口不满足条件/满足条件需要优化）
3. **何时更新答案**: 根据题目求最大还是最小决定

LeetCode Hot100 子串与数组专题速记

---

53. 最大子数组和 【中等】

思路

动态规划 / Kadane算法

核心思想：对于每个位置，要么延续之前的子数组，要么从当前位置重新开始。

状态转移：`dp[i] = max(dp[i-1] + nums[i], nums[i])`

简化理解：如果前面的累加和是负数，就丢弃；否则继续累加。

代码

```python
class Solution:
    def maxSubArray(self, nums: List[int]) -> int:
        # current_sum: 以当前位置结尾的最大子数组和
        # max_sum: 全局最大子数组和
        current_sum = max_sum = nums[0]

        for num in nums[1:]:
            # 如果前面累加和为负，从当前位置重新开始
            current_sum = max(current_sum + num, num)
            # 更新全局最大值
            max_sum = max(max_sum, current_sum)

        return max_sum
```

---

56. 合并区间 【中等】

思路

排序 + 贪心

1. 按区间起点排序
2. 遍历区间，判断当前区间与结果中最后一个区间是否重叠
3. 重叠则合并（更新右端点），不重叠则直接加入结果

关键判断：`当前起点 <= 上一个终点` 说明重叠

代码

```python
class Solution:
    def merge(self, intervals: List[List[int]]) -> List[List[int]]:
        # 按区间起点排序
        intervals.sort(key=lambda x: x[0])

        result = []
        for interval in intervals:
            # 如果结果为空，或当前区间与上一个不重叠
            if not result or interval[0] > result[-1][1]:
                result.append(interval)
            else:
                # 重叠，合并区间（更新右端点）
                result[-1][1] = max(result[-1][1], interval[1])

        return result
```

---

189. 轮转数组 【中等】

思路

三次翻转法

将数组右移k位等价于：

1. 整体翻转
2. 翻转前k个
3. 翻转后n-k个

例如 `[1,2,3,4,5,6,7]`，k=3:

- 整体翻转: `[7,6,5,4,3,2,1]`
- 前3翻转: `[5,6,7,4,3,2,1]`
- 后4翻转: `[5,6,7,1,2,3,4]`

注意：k可能大于n，需要取模

代码

```python
class Solution:
    def rotate(self, nums: List[int], k: int) -> None:
        n = len(nums)
        k = k % n  # 处理k大于数组长度的情况

        def reverse(left: int, right: int) -> None:
            """原地翻转数组的[left, right]部分"""
            while left < right:
                nums[left], nums[right] = nums[right], nums[left]
                left += 1
                right -= 1

        # 三次翻转
```

```
        reverse(0, n - 1)       # 整体翻转
        reverse(0, k - 1)       # 翻转前k个
        reverse(k, n - 1)       # 翻转后n-k个
```

---

238. 除自身以外数组的乘积 【中等】

思路

前缀积 + 后缀积

对于位置i：`answer[i] = 左边所有数的乘积 × 右边所有数的乘积`

优化空间：

1. 第一次遍历：answer存储左侧前缀积
2. 第二次遍历：用一个变量累积右侧后缀积，边遍历边更新answer

代码

```python
class Solution:
    def productExceptSelf(self, nums: List[int]) -> List[int]:
        n = len(nums)
        answer = [1] * n

        # 第一遍：计算每个位置左侧的乘积
        left_product = 1
        for i in range(n):
            answer[i] = left_product
            left_product *= nums[i]

        # 第二遍：计算每个位置右侧的乘积，同时更新答案
        right_product = 1
        for i in range(n - 1, -1, -1):
            answer[i] *= right_product
            right_product *= nums[i]

        return answer
```

---

41. 缺失的第一个正数 【困难】

思路

原地哈希（数组当哈希表）

核心思想：长度为n的数组，缺失的第一个正数一定在[1, n+1]范围内。

将数组本身当作哈希表：把数字x放到索引x-1的位置。

遍历后，第一个 `nums[i] != i+1` 的位置i，答案就是i+1。

交换条件：

1. nums[i]在[1, n]范围内

2. nums[i]不在正确位置（避免死循环）

代码

```
class Solution:
    def firstMissingPositive(self, nums: List[int]) -> int:
        n = len(nums)

        # 原地哈希：把数字x放到索引x-1的位置
        for i in range(n):
            # 当前数在有效范围内，且不在正确位置时，交换
            while 1 <= nums[i] <= n and nums[nums[i] - 1] != nums[i]:
                # 把nums[i]放到正确位置nums[i]-1
                correct_idx = nums[i] - 1
                nums[i], nums[correct_idx] = nums[correct_idx], nums[i]

        # 找第一个不在正确位置的数
        for i in range(n):
            if nums[i] != i + 1:
                return i + 1

        # 所有数都在正确位置，返回n+1
        return n + 1
```

速记总结

题目	核心算法	关键点
53. 最大子数组和	Kadane算法	前缀和为负就丢弃
56. 合并区间	排序+贪心	按起点排序，重叠就合并
189. 轮转数组	三次翻转	整体翻转→前k翻转→后n-k翻转
238. 除自身以外乘积	前缀积×后缀积	两次遍历，O(1)空间
41. 缺失第一个正数	原地哈希	数组当哈希表，x放到x-1位置

LeetCode Hot100 - 矩阵专题

---

73. 矩阵置零 【中等】

思路

核心：用第一行和第一列作为标记数组

1. 先记录第一行、第一列本身是否需要置零
2. 遍历矩阵（除第一行第一列），若 `matrix[i][j] == 0`，则标记 `matrix[i][0] = 0` 和 `matrix[0][j] = 0`
3. 根据标记，将对应行列置零（从后往前处理，避免覆盖标记）
4. 最后处理第一行和第一列

时间 O(mn)，空间 O(1)

代码

```
class Solution:
    def setZeroes(self, matrix: List[List[int]]) -> None:
        m, n = len(matrix), len(matrix[0])

        # 记录第一行、第一列是否需要置零
        first_row_zero = any(matrix[0][j] == 0 for j in range(n))
        first_col_zero = any(matrix[i][0] == 0 for i in range(m))

        # 用第一行第一列作为标记
        for i in range(1, m):
            for j in range(1, n):
                if matrix[i][j] == 0:
                    matrix[i][0] = 0  # 标记该行
                    matrix[0][j] = 0  # 标记该列

        # 根据标记置零（不含第一行第一列）
        for i in range(1, m):
            for j in range(1, n):
                if matrix[i][0] == 0 or matrix[0][j] == 0:
                    matrix[i][j] = 0

        # 处理第一行
        if first_row_zero:
            for j in range(n):
                matrix[0][j] = 0

        # 处理第一列
        if first_col_zero:
            for i in range(m):
                matrix[i][0] = 0
```

---

54. 螺旋矩阵 【中等】

思路

核心：定义四个边界，按层顺时针收缩

1. 定义边界：`top, bottom, left, right`
2. 按顺序遍历：右 → 下 → 左 → 上
3. 每遍历完一条边，收缩对应边界
4. 当边界交叉时停止

时间 O(mn)，空间 O(1)（不算结果数组）

代码

```python
class Solution:
    def spiralOrder(self, matrix: List[List[int]]) -> List[int]:
        if not matrix:
            return []

        res = []
        top, bottom = 0, len(matrix) - 1
        left, right = 0, len(matrix[0]) - 1

        while top <= bottom and left <= right:
            # 向右：遍历上边界
            for j in range(left, right + 1):
                res.append(matrix[top][j])
            top += 1

            # 向下：遍历右边界
            for i in range(top, bottom + 1):
                res.append(matrix[i][right])
            right -= 1

            # 向左：遍历下边界（注意判断是否还有行）
            if top <= bottom:
                for j in range(right, left - 1, -1):
                    res.append(matrix[bottom][j])
                bottom -= 1

            # 向上：遍历左边界（注意判断是否还有列）
            if left <= right:
                for i in range(bottom, top - 1, -1):
                    res.append(matrix[i][left])
                left += 1

        return res
```

---

48. 旋转图像 【中等】

思路

核心：先转置，再水平翻转

顺时针旋转 90° = 转置 + 左右翻转

- **转置**：`matrix[i][j]` 与 `matrix[j][i]` 交换
- **水平翻转**：每行反转

技巧记忆：

- 顺时针 90°：转置 + 左右翻转
- 逆时针 90°：转置 + 上下翻转

时间 $O(n^2)$，空间 $O(1)$

代码

```python
class Solution:
    def rotate(self, matrix: List[List[int]]) -> None:
        n = len(matrix)

        # 第一步：转置矩阵（沿主对角线翻转）
        for i in range(n):
            for j in range(i + 1, n):  # 只遍历上三角
                matrix[i][j], matrix[j][i] = matrix[j][i], matrix[i][j]

        # 第二步：水平翻转每一行
        for i in range(n):
            matrix[i].reverse()
            # 或者：matrix[i] = matrix[i][::-1]
```

---

240. 搜索二维矩阵 II 【中等】

思路

核心：从右上角或左下角开始搜索

矩阵特点：每行从左到右递增，每列从上到下递增

从右上角 (0, n-1) 开始：

- 当前值 > target：左移（排除当前列）
- 当前值 < target：下移（排除当前行）
- 相等则找到

为什么选右上角？ 因为它是行最大、列最小的位置，可以决定唯一方向

时间 O(m+n)，空间 O(1)

代码

```python
class Solution:
    def searchMatrix(self, matrix: List[List[int]], target: int) -> bool:
        if not matrix or not matrix[0]:
            return False

        m, n = len(matrix), len(matrix[0])

        # 从右上角开始
        i, j = 0, n - 1

        while i < m and j >= 0:
            if matrix[i][j] == target:
                return True
            elif matrix[i][j] > target:
                j -= 1  # 当前值太大，左移
```

```
            else:
                i += 1  # 当前值太小，下移

        return False
```

总结

题目	核心技巧	关键点
73. 矩阵置零	原地标记	用首行首列做标记，先记录首行首列状态
54. 螺旋矩阵	边界收缩	四个边界，顺时针遍历，注意边界判断
48. 旋转图像	转置+翻转	顺时针90°=转置+左右翻转
240. 搜索二维矩阵II	右上角搜索	利用有序性，每次排除一行或一列

LeetCode Hot100 链表专题速记

160. 相交链表 【简单】

思路：双指针法。指针 A 走完链表 A 后走链表 B，指针 B 走完链表 B 后走链表 A。若相交，两指针会在交点相遇（走过相同距离）；若不相交，都会走到 None。

```
class Solution:
    def getIntersectionNode(self, headA: ListNode, headB: ListNode) -> Optional[ListNode]:
        if not headA or not headB:
            return None

        pa, pb = headA, headB
        # 两指针分别走完两条链表，最终会在交点或None相遇
        while pa != pb:
            pa = pa.next if pa else headB
            pb = pb.next if pb else headA

        return pa
```

---

206. 反转链表 【简单】

思路：迭代法，用 prev 和 curr 两个指针，逐个反转节点的 next 指向。

```
class Solution:
    def reverseList(self, head: Optional[ListNode]) -> Optional[ListNode]:
        prev, curr = None, head

        while curr:
            next_temp = curr.next  # 保存下一个节点
            curr.next = prev       # 反转指向
            prev = curr            # prev 前进
```

```
            curr = next_temp        # curr 前进

        return prev
```

---

234. 回文链表 【简单】

思路：快慢指针找中点 + 反转后半部分 + 比较前后两半。空间 O(1)。

```
class Solution:
    def isPalindrome(self, head: Optional[ListNode]) -> bool:
        if not head or not head.next:
            return True

        # 1. 快慢指针找中点
        slow, fast = head, head
        while fast.next and fast.next.next:
            slow = slow.next
            fast = fast.next.next

        # 2. 反转后半部分（slow.next 开始）
        second_half = self.reverse(slow.next)

        # 3. 比较前后两半
        p1, p2 = head, second_half
        result = True
        while p2:  # 后半部分可能更短
            if p1.val != p2.val:
                result = False
                break
            p1 = p1.next
            p2 = p2.next

        return result

    def reverse(self, head):
        prev, curr = None, head
        while curr:
            next_temp = curr.next
            curr.next = prev
            prev = curr
            curr = next_temp
        return prev
```

---

141. 环形链表 【简单】

思路：快慢指针。快指针每次走2步，慢指针每次走1步。若有环，快慢指针一定会相遇；若无环，快指针先到达 None。

```
class Solution:
    def hasCycle(self, head: Optional[ListNode]) -> bool:
```

```
        if not head or not head.next:
            return False

        slow, fast = head, head

        while fast and fast.next:
            slow = slow.next
            fast = fast.next.next
            if slow == fast:  # 相遇说明有环
                return True

        return False
```

---

142. 环形链表 II 【中等】

思路：快慢指针相遇后，将一个指针移到头部，两指针同速前进，再次相遇点即为环入口。数学证明：设头到入口距离 a，入口到相遇点 b，相遇点再到入口 c，则 a = c。

```
class Solution:
    def detectCycle(self, head: Optional[ListNode]) -> Optional[ListNode]:
        if not head or not head.next:
            return None

        slow, fast = head, head

        # 第一次相遇
        while fast and fast.next:
            slow = slow.next
            fast = fast.next.next
            if slow == fast:
                break
        else:
            return None  # 无环

        # 一个指针回到头部，同速前进
        slow = head
        while slow != fast:
            slow = slow.next
            fast = fast.next

        return slow
```

---

21. 合并两个有序链表 【简单】

思路：用哨兵节点简化操作，比较两链表当前节点，较小者接入结果链表。

```
class Solution:
    def mergeTwoLists(self, list1: Optional[ListNode], list2: Optional[ListNode]) ->
Optional[ListNode]:
```

```
        dummy = ListNode(-1)  # 哨兵节点
        curr = dummy

        while list1 and list2:
            if list1.val <= list2.val:
                curr.next = list1
                list1 = list1.next
            else:
                curr.next = list2
                list2 = list2.next
            curr = curr.next

        # 接上剩余部分
        curr.next = list1 if list1 else list2

        return dummy.next
```

---

2. 两数相加 【中等】

思路：模拟加法，逐位相加并处理进位。用哨兵节点简化头节点处理。

```
class Solution:
    def addTwoNumbers(self, l1: Optional[ListNode], l2: Optional[ListNode]) ->
Optional[ListNode]:
        dummy = ListNode(0)
        curr = dummy
        carry = 0  # 进位

        while l1 or l2 or carry:
            val1 = l1.val if l1 else 0
            val2 = l2.val if l2 else 0

            total = val1 + val2 + carry
            carry = total // 10
            curr.next = ListNode(total % 10)
            curr = curr.next

            if l1: l1 = l1.next
            if l2: l2 = l2.next

        return dummy.next
```

---

19. 删除链表的倒数第 N 个结点 【中等】

思路：快慢指针，快指针先走 n 步，然后快慢同时走，当快指针到末尾时，慢指针指向待删除节点的前一个节点。使用哨兵节点处理删除头节点的情况。

```
class Solution:
    def removeNthFromEnd(self, head: Optional[ListNode], n: int) -> Optional[ListNode]:
```

```
        dummy = ListNode(0, head)
        fast = slow = dummy

        # 快指针先走 n+1 步
        for _ in range(n + 1):
            fast = fast.next

        # 同时前进，直到 fast 到末尾
        while fast:
            fast = fast.next
            slow = slow.next

        # 删除 slow 的下一个节点
        slow.next = slow.next.next

        return dummy.next
```

---

24. 两两交换链表中的节点 【中等】

思路：迭代法，每次处理两个节点的交换。需要记录前一对的尾节点，用于连接交换后的头节点。

```
class Solution:
    def swapPairs(self, head: Optional[ListNode]) -> Optional[ListNode]:
        dummy = ListNode(0, head)
        prev = dummy

        while prev.next and prev.next.next:
            # 定义要交换的两个节点
            first = prev.next
            second = prev.next.next

            # 交换
            first.next = second.next
            second.next = first
            prev.next = second

            # 移动 prev 到下一对的前面
            prev = first

        return dummy.next
```

---

25. K 个一组翻转链表 【困难】

思路：分组处理，每 K 个节点为一组进行反转。先检查剩余节点是否足够 K 个，不足则保持原样。用哨兵节点和辅助函数简化代码。

```
class Solution:
    def reverseKGroup(self, head: Optional[ListNode], k: int) -> Optional[ListNode]:
        dummy = ListNode(0, head)
```

```
        prev_group_end = dummy

        while True:
            # 检查是否还有 k 个节点
            kth = self.getKth(prev_group_end, k)
            if not kth:
                break

            next_group_start = kth.next

            # 反转当前组 [prev_group_end.next, kth]
            prev, curr = kth.next, prev_group_end.next
            while curr != next_group_start:
                next_temp = curr.next
                curr.next = prev
                prev = curr
                curr = next_temp

            # 连接：原来的第一个节点变成最后一个
            first_in_group = prev_group_end.next
            prev_group_end.next = kth
            prev_group_end = first_in_group

        return dummy.next

    def getKth(self, start, k):
        """从 start 开始，返回第 k 个节点"""
        curr = start
        for _ in range(k):
            if not curr:
                return None
            curr = curr.next
        return curr
```

---

138. 随机链表的复制 【中等】

思路：方法一：哈希表存储原节点到新节点的映射。方法二：交织链表法（空间O(1)）。这里用哈希表法，更直观。

```
class Solution:
    def copyRandomList(self, head: 'Optional[Node]') -> 'Optional[Node]':
        if not head:
            return None

        # 哈希表：原节点 -> 新节点
        old_to_new = {}

        # 第一遍：创建所有新节点
        curr = head
        while curr:
```

```
            old_to_new[curr] = Node(curr.val)
            curr = curr.next

        # 第二遍：连接 next 和 random
        curr = head
        while curr:
            new_node = old_to_new[curr]
            new_node.next = old_to_new.get(curr.next)
            new_node.random = old_to_new.get(curr.random)
            curr = curr.next

        return old_to_new[head]
```

---

148. 排序链表 【中等】

思路：归并排序。快慢指针找中点，递归排序左右两半，然后合并。时间 O(nlogn)，空间 O(logn)（递归栈）。

```
class Solution:
    def sortList(self, head: Optional[ListNode]) -> Optional[ListNode]:
        # 基准情况：空或单节点
        if not head or not head.next:
            return head

        # 快慢指针找中点（slow 停在前半部分的最后一个节点）
        slow, fast = head, head.next
        while fast and fast.next:
            slow = slow.next
            fast = fast.next.next

        # 断开链表
        mid = slow.next
        slow.next = None

        # 递归排序左右两半
        left = self.sortList(head)
        right = self.sortList(mid)

        # 合并两个有序链表
        return self.merge(left, right)

    def merge(self, l1, l2):
        dummy = ListNode(0)
        curr = dummy
        while l1 and l2:
            if l1.val <= l2.val:
                curr.next = l1
                l1 = l1.next
            else:
                curr.next = l2
```

```
                l2 = l2.next
            curr = curr.next
        curr.next = l1 if l1 else l2
        return dummy.next
```

---

23. 合并 K 个升序链表 【困难】

思路：最小堆（优先队列）。将所有链表的头节点入堆，每次取出最小节点，将其下一个节点入堆。时间 O(nlogk)，k 为链表数量。

```
import heapq

class Solution:
    def mergeKLists(self, lists: List[Optional[ListNode]]) -> Optional[ListNode]:
        dummy = ListNode(0)
        curr = dummy

        # 最小堆：(节点值, 唯一序号, 节点) - 序号用于值相同时比较
        heap = []
        for i, node in enumerate(lists):
            if node:
                heapq.heappush(heap, (node.val, i, node))

        idx = len(lists)  # 用于生成唯一序号

        while heap:
            val, _, node = heapq.heappop(heap)
            curr.next = node
            curr = curr.next

            if node.next:
                heapq.heappush(heap, (node.next.val, idx, node.next))
                idx += 1

        return dummy.next
```

---

146. LRU 缓存 【中等】

思路：哈希表 + 双向链表。哈希表实现 O(1) 查找，双向链表维护访问顺序（最近访问的在头部，最久未访问的在尾部）。

```
class DLinkedNode:
    def __init__(self, key=0, value=0):
        self.key = key
        self.value = value
        self.prev = None
        self.next = None

class LRUCache:
    def __init__(self, capacity: int):
```

```python
        self.capacity = capacity
        self.cache = {}  # key -> DLinkedNode
        # 使用伪头部和伪尾部简化边界处理
        self.head = DLinkedNode()
        self.tail = DLinkedNode()
        self.head.next = self.tail
        self.tail.prev = self.head

    def get(self, key: int) -> int:
        if key not in self.cache:
            return -1
        node = self.cache[key]
        self._move_to_head(node)  # 移到头部表示最近访问
        return node.value

    def put(self, key: int, value: int) -> None:
        if key in self.cache:
            node = self.cache[key]
            node.value = value
            self._move_to_head(node)
        else:
            node = DLinkedNode(key, value)
            self.cache[key] = node
            self._add_to_head(node)
            if len(self.cache) > self.capacity:
                # 删除尾部节点（最久未使用）
                removed = self._remove_tail()
                del self.cache[removed.key]

    def _add_to_head(self, node):
        node.prev = self.head
        node.next = self.head.next
        self.head.next.prev = node
        self.head.next = node

    def _remove_node(self, node):
        node.prev.next = node.next
        node.next.prev = node.prev

    def _move_to_head(self, node):
        self._remove_node(node)
        self._add_to_head(node)

    def _remove_tail(self):
        node = self.tail.prev
        self._remove_node(node)
        return node
```

总结

题目	核心技巧
160. 相交链表	双指针走等长路径
206. 反转链表	迭代反转，prev/curr 双指针
234. 回文链表	快慢指针找中点 + 反转后半
141. 环形链表	快慢指针判环
142. 环形链表 II	快慢指针 + 数学推导找入口
21. 合并两个有序链表	哨兵节点 + 双指针
2. 两数相加	模拟加法，处理进位
19. 删除倒数第N个	快慢指针，快先走N步
24. 两两交换	迭代，注意连接顺序
25. K个一组翻转	分组反转 + 连接
138. 随机链表复制	哈希表映射
148. 排序链表	归并排序
23. 合并K个链表	最小堆
146. LRU缓存	哈希表 + 双向链表

LeetCode Hot100 二叉树专题速记

94. 二叉树的中序遍历 【简单】

思路：中序遍历顺序为「左-根-右」。递归法直接按顺序访问；迭代法用栈模拟，先一路向左入栈，出栈时访问节点并转向右子树。

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right

class Solution:
    def inorderTraversal(self, root: Optional[TreeNode]) -> List[int]:
        res = []
        stack = []
        cur = root

        while cur or stack:
            # 一路向左，全部入栈
            while cur:
                stack.append(cur)
```

```
            cur = cur.left
        # 出栈，访问节点
        cur = stack.pop()
        res.append(cur.val)
        # 转向右子树
        cur = cur.right

    return res
```

---

104. 二叉树的最大深度 【简单】

思路：递归：当前节点深度 = 1 + max(左子树深度, 右子树深度)。空节点返回0。

```
class Solution:
    def maxDepth(self, root: Optional[TreeNode]) -> int:
        if not root:
            return 0
        # 递归计算左右子树深度，取较大值+1
        return 1 + max(self.maxDepth(root.left), self.maxDepth(root.right))
```

---

226. 翻转二叉树 【简单】

思路：递归交换每个节点的左右子树。先递归翻转左右子树，再交换左右子节点。

```
class Solution:
    def invertTree(self, root: Optional[TreeNode]) -> Optional[TreeNode]:
        if not root:
            return None
        # 递归翻转左右子树
        left = self.invertTree(root.left)
        right = self.invertTree(root.right)
        # 交换左右子节点
        root.left, root.right = right, left
        return root
```

---

101. 对称二叉树 【简单】

思路：判断左右子树是否镜像对称。递归比较：左子树的左 vs 右子树的右，左子树的右 vs 右子树的左。

```
class Solution:
    def isSymmetric(self, root: Optional[TreeNode]) -> bool:
        def isMirror(left: TreeNode, right: TreeNode) -> bool:
            # 都为空，对称
            if not left and not right:
                return True
            # 一个为空一个不为空，不对称
            if not left or not right:
                return False
```

```
            # 值相等且子树镜像对称
            return (left.val == right.val and
                    isMirror(left.left, right.right) and
                    isMirror(left.right, right.left))

        return isMirror(root.left, root.right) if root else True
```

---

543. 二叉树的直径 【简单】

思路：直径 = 某节点左子树深度 + 右子树深度的最大值。在求深度的过程中更新全局最大直径。

```
class Solution:
    def diameterOfBinaryTree(self, root: Optional[TreeNode]) -> int:
        self.diameter = 0

        def depth(node: TreeNode) -> int:
            if not node:
                return 0
            left = depth(node.left)
            right = depth(node.right)
            # 更新直径：经过当前节点的路径长度
            self.diameter = max(self.diameter, left + right)
            # 返回当前节点的深度
            return 1 + max(left, right)

        depth(root)
        return self.diameter
```

---

102. 二叉树的层序遍历 【中等】

思路：BFS，用队列逐层遍历。每次处理一层，记录当前层节点数，依次出队并将子节点入队。

```
from collections import deque

class Solution:
    def levelOrder(self, root: Optional[TreeNode]) -> List[List[int]]:
        if not root:
            return []

        res = []
        queue = deque([root])

        while queue:
            level = []
            size = len(queue)  # 当前层的节点数
            for _ in range(size):
                node = queue.popleft()
                level.append(node.val)
                # 子节点入队
```

```
                if node.left:
                    queue.append(node.left)
                if node.right:
                    queue.append(node.right)
            res.append(level)

        return res
```

---

108. 将有序数组转换为二叉搜索树 【简单】

思路：有序数组的中间元素作为根节点，左半部分构建左子树，右半部分构建右子树，递归构建平衡BST。

```
class Solution:
    def sortedArrayToBST(self, nums: List[int]) -> Optional[TreeNode]:
        def build(left: int, right: int) -> TreeNode:
            if left > right:
                return None
            # 选择中间元素作为根节点
            mid = (left + right) // 2
            node = TreeNode(nums[mid])
            # 递归构建左右子树
            node.left = build(left, mid - 1)
            node.right = build(mid + 1, right)
            return node

        return build(0, len(nums) - 1)
```

---

98. 验证二叉搜索树 【中等】

思路：BST的中序遍历是递增序列。可用递归验证每个节点值在(lower, upper)范围内，或中序遍历检查是否严格递增。

```
class Solution:
    def isValidBST(self, root: Optional[TreeNode]) -> bool:
        def validate(node: TreeNode, lower: float, upper: float) -> bool:
            if not node:
                return True
            # 当前节点值必须在(lower, upper)范围内
            if node.val <= lower or node.val >= upper:
                return False
            # 左子树值 < 当前值，右子树值 > 当前值
            return (validate(node.left, lower, node.val) and
                    validate(node.right, node.val, upper))

        return validate(root, float('-inf'), float('inf'))
```

---

230. 二叉搜索树中第 K 小的元素 【中等】

思路：BST中序遍历得到升序序列，第K个访问的就是第K小。用迭代法可以提前终止。

```python
class Solution:
    def kthSmallest(self, root: Optional[TreeNode], k: int) -> int:
        stack = []
        cur = root

        while cur or stack:
            # 一路向左
            while cur:
                stack.append(cur)
                cur = cur.left
            cur = stack.pop()
            k -= 1
            # 找到第k小
            if k == 0:
                return cur.val
            cur = cur.right

        return -1
```

---

199. 二叉树的右视图 【中等】

思路：层序遍历，每层最后一个节点即为右视图可见节点。或DFS优先访问右子树，记录每层第一个访问的节点。

```python
from collections import deque

class Solution:
    def rightSideView(self, root: Optional[TreeNode]) -> List[int]:
        if not root:
            return []

        res = []
        queue = deque([root])

        while queue:
            size = len(queue)
            for i in range(size):
                node = queue.popleft()
                # 每层最后一个节点加入结果
                if i == size - 1:
                    res.append(node.val)
                if node.left:
                    queue.append(node.left)
                if node.right:
                    queue.append(node.right)

        return res
```

---

114. 二叉树展开为链表 【中等】

思路：将左子树插入到右子树的位置，原右子树接到左子树最右节点。按前序遍历顺序依次处理每个节点。

```
class Solution:
    def flatten(self, root: Optional[TreeNode]) -> None:
        """
        Do not return anything, modify root in-place instead.
        """
        cur = root
        while cur:
            if cur.left:
                # 找到左子树的最右节点
                pre = cur.left
                while pre.right:
                    pre = pre.right
                # 将右子树接到左子树最右节点
                pre.right = cur.right
                # 左子树移到右边
                cur.right = cur.left
                cur.left = None
            # 处理下一个节点
            cur = cur.right
```

---

105. 从前序与中序遍历序列构造二叉树 【中等】

思路：前序第一个元素是根，在中序中找到根的位置，左边是左子树，右边是右子树，递归构建。用哈希表加速查找。

```
class Solution:
    def buildTree(self, preorder: List[int], inorder: List[int]) -> Optional[TreeNode]:
        # 哈希表存储中序遍历的索引
        idx_map = {val: idx for idx, val in enumerate(inorder)}

        def build(pre_left: int, pre_right: int, in_left: int, in_right: int) -> TreeNode:
            if pre_left > pre_right:
                return None

            # 前序遍历第一个元素是根节点
            root_val = preorder[pre_left]
            root = TreeNode(root_val)

            # 在中序遍历中找到根节点位置
            in_root = idx_map[root_val]
            # 左子树的节点数
            left_size = in_root - in_left

            # 递归构建左右子树
            root.left = build(pre_left + 1, pre_left + left_size, in_left, in_root - 1)
            root.right = build(pre_left + left_size + 1, pre_right, in_root + 1, in_right)
```

```
            return root

        return build(0, len(preorder) - 1, 0, len(inorder) - 1)
```

---

437. 路径总和 III 【中等】

思路：前缀和 + 哈希表。记录从根到当前节点的前缀和，路径和为targetSum等价于存在前缀和差为targetSum。

```
from collections import defaultdict

class Solution:
    def pathSum(self, root: Optional[TreeNode], targetSum: int) -> int:
        # 前缀和计数，初始化0出现1次
        prefix_count = defaultdict(int)
        prefix_count[0] = 1

        def dfs(node: TreeNode, curr_sum: int) -> int:
            if not node:
                return 0

            # 当前前缀和
            curr_sum += node.val
            # 找是否存在前缀和为 curr_sum - targetSum
            count = prefix_count[curr_sum - targetSum]

            # 记录当前前缀和
            prefix_count[curr_sum] += 1
            # 递归左右子树
            count += dfs(node.left, curr_sum)
            count += dfs(node.right, curr_sum)
            # 回溯，移除当前前缀和
            prefix_count[curr_sum] -= 1

            return count

        return dfs(root, 0)
```

---

236. 二叉树的最近公共祖先 【中等】

思路：递归查找p和q。若当前节点为p或q则返回当前节点；若p和q分别在左右子树，则当前节点为LCA；否则返回非空的那个子树结果。

```
class Solution:
    def lowestCommonAncestor(self, root: 'TreeNode', p: 'TreeNode', q: 'TreeNode') ->
'TreeNode':
        # 递归终止条件
        if not root or root == p or root == q:
            return root
```

```
        # 递归查找左右子树
        left = self.lowestCommonAncestor(root.left, p, q)
        right = self.lowestCommonAncestor(root.right, p, q)

        # p和q分别在左右子树，当前节点就是LCA
        if left and right:
            return root
        # 都在左子树或都在右子树
        return left if left else right
```

---

124. 二叉树中的最大路径和 【困难】

思路：对每个节点计算「经过该节点的最大路径和」，更新全局最大值。递归返回「以该节点为端点的最大贡献值」（只能选左或右分支）。

```
class Solution:
    def maxPathSum(self, root: Optional[TreeNode]) -> int:
        self.max_sum = float('-inf')

        def maxGain(node: TreeNode) -> int:
            if not node:
                return 0

            # 递归计算左右子树的最大贡献值（负数则取0，表示不选）
            left_gain = max(maxGain(node.left), 0)
            right_gain = max(maxGain(node.right), 0)

            # 经过当前节点的最大路径和
            path_sum = node.val + left_gain + right_gain
            # 更新全局最大值
            self.max_sum = max(self.max_sum, path_sum)

            # 返回以当前节点为端点的最大贡献值（只能选一边）
            return node.val + max(left_gain, right_gain)

        maxGain(root)
        return self.max_sum
```

速记要点总结

题目	核心技巧
中序遍历	栈 + 一路向左
最大深度	递归：1 + max(左, 右)
翻转二叉树	递归交换左右子树
对称二叉树	镜像比较：左左vs右右，左右vs右左

二叉树直径	求深度时更新直径 = 左深度 + 右深度
层序遍历	BFS + 队列 + 记录每层size
有序数组转BST	中间元素为根，递归建树
验证BST	递归传递(lower, upper)范围
第K小元素	中序遍历第K个
右视图	层序遍历取每层最后一个
展开为链表	左子树插右边，原右子树接左子树最右
前序+中序构建	前序首位是根，中序定左右范围
路径总和III	前缀和 + 哈希表 + 回溯
最近公共祖先	递归：左右都有则返回当前，否则返回非空
最大路径和	递归返回单边贡献，更新经过当前节点的路径和

LeetCode Hot100 图论专题速记

---

200. 岛屿数量 【中等】

思路

核心算法：DFS/BFS 连通分量计数

1. 遍历整个网格，遇到陆地 `'1'` 时，岛屿数量 +1
2. 从该位置启动 DFS/BFS，将所有相连的陆地标记为已访问（改为 `'0'` ）
3. 继续遍历，直到遍历完整个网格

关键点：

- 原地修改避免额外空间（将访问过的 `'1'` 改为 `'0'` ）
- 四个方向扩展：上、下、左、右

代码

```
class Solution:
    def numIslands(self, grid: List[List[str]]) -> int:
        if not grid:
            return 0

        m, n = len(grid), len(grid[0])
        count = 0

        def dfs(i, j):
            # 边界检查 或 遇到水
            if i < 0 or i >= m or j < 0 or j >= n or grid[i][j] == '0':
                return
            # 标记为已访问
```

```
            grid[i][j] = '0'
            # 向四个方向扩展
            dfs(i + 1, j)
            dfs(i - 1, j)
            dfs(i, j + 1)
            dfs(i, j - 1)

        for i in range(m):
            for j in range(n):
                if grid[i][j] == '1':
                    count += 1  # 发现新岛屿
                    dfs(i, j)   # 淹没整个岛屿

        return count
```

---

994. 腐烂的橘子 【中等】

思路

核心算法：多源 BFS

1. 统计新鲜橘子数量，将所有腐烂橘子入队（多源起点）
2. BFS 层序遍历，每一层代表过了 1 分钟
3. 腐烂橘子感染相邻的新鲜橘子，新鲜橘子数量减少
4. 遍历完成后，若还有新鲜橘子，返回 -1；否则返回经过的时间

关键点：

- 多源 BFS：所有腐烂橘子同时开始扩散
- 按层遍历记录时间（分钟数）

代码

```
class Solution:
    def orangesRotting(self, grid: List[List[int]]) -> int:
        from collections import deque

        m, n = len(grid), len(grid[0])
        queue = deque()
        fresh = 0

        # 统计新鲜橘子，收集腐烂橘子位置
        for i in range(m):
            for j in range(n):
                if grid[i][j] == 2:
                    queue.append((i, j))
                elif grid[i][j] == 1:
                    fresh += 1

        # 没有新鲜橘子，直接返回 0
        if fresh == 0:
            return 0
```

```
        minutes = 0
        directions = [(0, 1), (0, -1), (1, 0), (-1, 0)]

        # BFS 层序遍历
        while queue:
            minutes += 1
            for _ in range(len(queue)):  # 当前层的所有腐烂橘子
                x, y = queue.popleft()
                for dx, dy in directions:
                    nx, ny = x + dx, y + dy
                    # 感染相邻的新鲜橘子
                    if 0 <= nx < m and 0 <= ny < n and grid[nx][ny] == 1:
                        grid[nx][ny] = 2
                        fresh -= 1
                        queue.append((nx, ny))

        # 减去多计算的一轮（最后一轮没有新感染）
        return minutes - 1 if fresh == 0 else -1
```

---

207. 课程表 【中等】

思路

核心算法：拓扑排序（BFS / Kahn 算法）

1. 建立邻接表和入度数组
2. 将所有入度为 0 的节点入队（无前置课程）
3. BFS 遍历：弹出节点，将其指向的节点入度 -1
4. 若入度减为 0，则入队
5. 最终检查是否所有课程都被处理（有环则无法完成）

关键点：

- 有向图判环问题 = 拓扑排序
- 能完成拓扑排序（处理 n 个节点）= 无环 = 能修完所有课

代码

```
class Solution:
    def canFinish(self, numCourses: int, prerequisites: List[List[int]]) -> bool:
        from collections import deque

        # 建图：邻接表 + 入度数组
        indegree = [0] * numCourses
        adj = [[] for _ in range(numCourses)]

        for cur, pre in prerequisites:
            adj[pre].append(cur)  # pre -> cur
            indegree[cur] += 1

        # 入度为 0 的节点入队
```

```
        queue = deque([i for i in range(numCourses) if indegree[i] == 0])
        count = 0  # 已处理的课程数

        while queue:
            course = queue.popleft()
            count += 1
            # 将该课程指向的课程入度 -1
            for next_course in adj[course]:
                indegree[next_course] -= 1
                if indegree[next_course] == 0:
                    queue.append(next_course)

        # 能处理所有课程 = 无环
        return count == numCourses
```

---

208. 实现 Trie (前缀树) 【中等】

思路

核心算法：前缀树（字典树）

1. 每个节点包含：子节点映射（字典）+ 是否为单词结尾标记
2. `insert`：逐字符创建/遍历节点，最后标记结束
3. `search`：逐字符遍历，检查是否存在且是单词结尾
4. `startsWith`：逐字符遍历，只检查前缀是否存在

关键点：

- 用字典存储子节点，key 为字符，value 为子节点
- `is_end` 标记区分完整单词和前缀

代码

```
class Trie:

    def __init__(self):
        self.children = {}  # 子节点映射
        self.is_end = False  # 是否为单词结尾

    def insert(self, word: str) -> None:
        node = self
        for char in word:
            if char not in node.children:
                node.children[char] = Trie()  # 创建新节点
            node = node.children[char]
        node.is_end = True  # 标记单词结束

    def search(self, word: str) -> bool:
        node = self._search_prefix(word)
        return node is not None and node.is_end
```

```python
    def startsWith(self, prefix: str) -> bool:
        return self._search_prefix(prefix) is not None

    def _search_prefix(self, prefix: str):
        """查找前缀对应的节点"""
        node = self
        for char in prefix:
            if char not in node.children:
                return None
            node = node.children[char]
        return node


# Your Trie object will be instantiated and called as such:
# obj = Trie()
# obj.insert(word)
# param_2 = obj.search(word)
# param_3 = obj.startsWith(prefix)
```

总结

题目	核心算法	关键技巧
200. 岛屿数量	DFS/BFS	原地标记，连通分量计数
994. 腐烂的橘子	多源 BFS	层序遍历记录时间
207. 课程表	拓扑排序	入度数组 + BFS
208. 实现 Trie	前缀树	字典存子节点 + 结束标记

LeetCode Hot100 回溯专题速记

> *回溯算法核心模板：*
>
> *1. 做选择 -> 2. 递归 -> 3. 撤销选择 关键：理解什么时候收集结果，什么时候剪枝*

---

46. 全排列 【中等】

思路

- 经典回溯模板题，每次从未使用的数字中选一个加入路径
- 使用 `used` 数组标记已使用的数字
- 当路径长度等于数组长度时，收集结果

代码

```python
class Solution:
    def permute(self, nums: List[int]) -> List[List[int]]:
        res = []
        path = []
```

```
        used = [False] * len(nums)

        def backtrack():
            # 收集结果：路径长度等于数组长度
            if len(path) == len(nums):
                res.append(path[:])  # 注意要拷贝
                return

            for i in range(len(nums)):
                if used[i]:  # 跳过已使用的数字
                    continue
                # 做选择
                path.append(nums[i])
                used[i] = True
                # 递归
                backtrack()
                # 撤销选择
                path.pop()
                used[i] = False

        backtrack()
        return res
```

---

78. 子集 【中等】

思路

- 子集问题：每个元素都有「选」或「不选」两种状态
- 方法一：每层递归都是一个有效子集，直接收集
- 方法二：用 start 控制遍历起点，避免重复

代码

```
class Solution:
    def subsets(self, nums: List[int]) -> List[List[int]]:
        res = []
        path = []

        def backtrack(start):
            # 每个节点都是一个子集，直接收集
            res.append(path[:])

            for i in range(start, len(nums)):
                path.append(nums[i])  # 做选择
                backtrack(i + 1)       # 递归，注意是 i+1 不是 start+1
                path.pop()             # 撤销选择

        backtrack(0)
        return res
```

---

17. 电话号码的字母组合 【中等】

思路

- 多个集合的组合问题
- 用哈希表存储数字到字母的映射
- 每层递归处理一个数字，遍历该数字对应的所有字母

代码

```
class Solution:
    def letterCombinations(self, digits: str) -> List[str]:
        if not digits:
            return []

        # 数字到字母的映射
        phone = {
            '2': 'abc', '3': 'def', '4': 'ghi', '5': 'jkl',
            '6': 'mno', '7': 'pqrs', '8': 'tuv', '9': 'wxyz'
        }

        res = []

        def backtrack(index, path):
            # 收集结果：处理完所有数字
            if index == len(digits):
                res.append(''.join(path))
                return

            # 获取当前数字对应的字母
            letters = phone[digits[index]]
            for letter in letters:
                path.append(letter)      # 做选择
                backtrack(index + 1, path)  # 递归下一个数字
                path.pop()               # 撤销选择

        backtrack(0, [])
        return res
```

---

39. 组合总和 【中等】

思路

- 同一个数字可以无限制重复选取
- 用 start 避免重复组合（如 [2,2,3] 和 [2,3,2]）
- 剪枝：当前和超过 target 时直接返回

代码

```
class Solution:
    def combinationSum(self, candidates: List[int], target: int) -> List[List[int]]:
```

```
        res = []
        path = []

        def backtrack(start, remain):
            # 找到目标
            if remain == 0:
                res.append(path[:])
                return
            # 剪枝：超过目标
            if remain < 0:
                return

            for i in range(start, len(candidates)):
                path.append(candidates[i])
                # 注意这里是 i 而不是 i+1，因为可以重复使用
                backtrack(i, remain - candidates[i])
                path.pop()

        backtrack(0, target)
        return res
```

---

22. 括号生成 【中等】

思路

- 左括号数量 < n 时可以加左括号
- 右括号数量 < 左括号数量时可以加右括号
- 当左右括号都用完时收集结果

代码

```
class Solution:
    def generateParenthesis(self, n: int) -> List[str]:
        res = []

        def backtrack(path, left, right):
            # 收集结果：左右括号都用完
            if left == n and right == n:
                res.append(''.join(path))
                return

            # 可以加左括号
            if left < n:
                path.append('(')
                backtrack(path, left + 1, right)
                path.pop()

            # 可以加右括号（右括号数量必须小于左括号）
            if right < left:
                path.append(')')
```

```
            backtrack(path, left, right + 1)
            path.pop()

    backtrack([], 0, 0)
    return res
```

---

79. 单词搜索 【中等】

思路

- 二维网格上的回溯搜索
- 从每个位置开始尝试匹配单词
- 用临时标记避免重复访问（原地修改或 visited 数组）
- 四个方向 DFS 搜索

代码

```
class Solution:
    def exist(self, board: List[List[str]], word: str) -> bool:
        m, n = len(board), len(board[0])

        def backtrack(i, j, k):
            # 匹配完成
            if k == len(word):
                return True
            # 越界或字符不匹配
            if i < 0 or i >= m or j < 0 or j >= n or board[i][j] != word[k]:
                return False

            # 标记已访问（原地修改，节省空间）
            temp = board[i][j]
            board[i][j] = '#'

            # 四个方向搜索
            found = (backtrack(i + 1, j, k + 1) or
                     backtrack(i - 1, j, k + 1) or
                     backtrack(i, j + 1, k + 1) or
                     backtrack(i, j - 1, k + 1))

            # 撤销标记
            board[i][j] = temp

            return found

        # 从每个位置开始尝试
        for i in range(m):
            for j in range(n):
                if backtrack(i, j, 0):
                    return True
        return False
```

---

131. 分割回文串 【中等】

思路

- 切割问题本质也是组合问题
- 用 start 表示切割起点，枚举切割终点
- 只有当前子串是回文时才继续递归
- 切到字符串末尾时收集结果

代码

```
class Solution:
    def partition(self, s: str) -> List[List[str]]:
        res = []
        path = []

        def is_palindrome(sub):
            return sub == sub[::-1]

        def backtrack(start):
            # 切到末尾，收集结果
            if start == len(s):
                res.append(path[:])
                return

            for end in range(start + 1, len(s) + 1):
                sub = s[start:end]
                # 剪枝：只有回文才继续
                if is_palindrome(sub):
                    path.append(sub)
                    backtrack(end)  # 从 end 开始继续切割
                    path.pop()

        backtrack(0)
        return res
```

---

51. N 皇后 【困难】

思路

- 逐行放置皇后，每行只能放一个
- 检查列冲突、主对角线冲突（行-列相同）、副对角线冲突（行+列相同）
- 用三个集合分别记录被占用的列、主对角线、副对角线

代码

```
class Solution:
    def solveNQueens(self, n: int) -> List[List[str]]:
        res = []
        board = [['.'] * n for _ in range(n)]
```

```
        cols = set()        # 被占用的列
        diag1 = set()       # 被占用的主对角线 (row - col)
        diag2 = set()       # 被占用的副对角线 (row + col)

        def backtrack(row):
            # 所有行都放置完成
            if row == n:
                res.append([''.join(r) for r in board])
                return

            for col in range(n):
                # 检查是否可以放置
                if col in cols or (row - col) in diag1 or (row + col) in diag2:
                    continue

                # 放置皇后
                board[row][col] = 'Q'
                cols.add(col)
                diag1.add(row - col)
                diag2.add(row + col)

                # 递归下一行
                backtrack(row + 1)

                # 撤销
                board[row][col] = '.'
                cols.remove(col)
                diag1.remove(row - col)
                diag2.remove(row + col)

        backtrack(0)
        return res
```

回溯总结

题型	关键点	代表题目
排列	used数组标记	46. 全排列
子集	start控制起点，每个节点都收集	78. 子集
组合	start控制起点，达到条件收集	39. 组合总和
切割	start表示切割位置	131. 分割回文串
棋盘	逐行/逐格搜索	51. N皇后, 79. 单词搜索

回溯模板：

```
def backtrack(路径, 选择列表):
    if 满足结束条件:
```

```
        收集结果
        return

    for 选择 in 选择列表:
        if 不合法:
            continue  # 剪枝
        做选择
        backtrack(新路径, 新选择列表)
        撤销选择
```

LeetCode Hot100 二分查找专题

35. 搜索插入位置 【简单】

思路

标准二分查找模板题。找到第一个 >= target 的位置。

- 如果找到 target，返回其索引
- 如果未找到，返回应该插入的位置（即 left 指针最终位置）

代码

```python
class Solution:
    def searchInsert(self, nums: List[int], target: int) -> int:
        left, right = 0, len(nums) - 1

        while left <= right:
            mid = left + (right - left) // 2  # 防止溢出
            if nums[mid] == target:
                return mid
            elif nums[mid] < target:
                left = mid + 1  # 目标在右半部分
            else:
                right = mid - 1  # 目标在左半部分

        # left 就是第一个 >= target 的位置
        return left
```

---

74. 搜索二维矩阵 【中等】

思路

将二维矩阵看作一维有序数组进行二分查找。

- 矩阵大小 m x n，总共 m*n 个元素
- 一维索引 idx 对应二维坐标：row = idx // n, col = idx % n

代码

```python
class Solution:
    def searchMatrix(self, matrix: List[List[int]], target: int) -> bool:
        m, n = len(matrix), len(matrix[0])
        left, right = 0, m * n - 1

        while left <= right:
            mid = left + (right - left) // 2
            # 将一维索引转换为二维坐标
            row, col = mid // n, mid % n
            val = matrix[row][col]

            if val == target:
                return True
            elif val < target:
                left = mid + 1
            else:
                right = mid - 1

        return False
```

---

34. 在排序数组中查找元素的第一个和最后一个位置 【中等】

思路

两次二分查找：

1. 第一次找左边界（第一个等于 target 的位置）
2. 第二次找右边界（最后一个等于 target 的位置）

关键：找左边界时，即使找到 target 也继续向左收缩；找右边界同理。

代码

```python
class Solution:
    def searchRange(self, nums: List[int], target: int) -> List[int]:
        def findLeft(nums, target):
            """找第一个等于 target 的位置"""
            left, right = 0, len(nums) - 1
            while left <= right:
                mid = left + (right - left) // 2
                if nums[mid] < target:
                    left = mid + 1
                else:
                    right = mid - 1  # nums[mid] >= target，继续向左找
            # 检查 left 是否越界以及是否等于 target
            if left < len(nums) and nums[left] == target:
                return left
            return -1

        def findRight(nums, target):
```

```
        """找最后一个等于 target 的位置"""
        left, right = 0, len(nums) - 1
        while left <= right:
            mid = left + (right - left) // 2
            if nums[mid] <= target:
                left = mid + 1  # nums[mid] <= target，继续向右找
            else:
                right = mid - 1
        # 检查 right 是否越界以及是否等于 target
        if right >= 0 and nums[right] == target:
            return right
        return -1

    return [findLeft(nums, target), findRight(nums, target)]
```

---

33. 搜索旋转排序数组 【中等】

思路

旋转后数组分成两段有序部分。每次二分时，判断 mid 在哪一段：

1. 若 nums[left] <= nums[mid]，说明左半部分有序
   - 若 target 在 [nums[left], nums[mid]) 内，在左半部分找
   - 否则在右半部分找
2. 否则右半部分有序，同理判断

代码

```
class Solution:
    def search(self, nums: List[int], target: int) -> int:
        left, right = 0, len(nums) - 1

        while left <= right:
            mid = left + (right - left) // 2

            if nums[mid] == target:
                return mid

            # 判断哪一半是有序的
            if nums[left] <= nums[mid]:  # 左半部分有序
                # 判断 target 是否在左半部分
                if nums[left] <= target < nums[mid]:
                    right = mid - 1
                else:
                    left = mid + 1
            else:  # 右半部分有序
                # 判断 target 是否在右半部分
                if nums[mid] < target <= nums[right]:
                    left = mid + 1
                else:
```

```
                    right = mid - 1

        return -1
```

---

153. 寻找旋转排序数组中的最小值 【中等】

思路

最小值是旋转点。比较 nums[mid] 与 nums[right]：

- 若 nums[mid] > nums[right]，最小值在右半部分（不含 mid）
- 若 nums[mid] <= nums[right]，最小值在左半部分（含 mid）

注意：这里用 nums[right] 而非 nums[left] 比较更简洁。

代码

```python
class Solution:
    def findMin(self, nums: List[int]) -> int:
        left, right = 0, len(nums) - 1

        while left < right:  # 注意：这里是 < 不是 <=
            mid = left + (right - left) // 2

            if nums[mid] > nums[right]:
                # 最小值在 mid 右边（不含 mid）
                left = mid + 1
            else:
                # 最小值在 mid 左边（含 mid）
                right = mid

        # left == right 时找到最小值
        return nums[left]
```

---

4. 寻找两个正序数组的中位数 【困难】

思路

核心思想：在较短数组上二分，将两个数组各分成左右两部分，使得：

1. 左部分元素个数 = 右部分元素个数（或多1个）
2. 左部分最大值 <= 右部分最小值

设在 nums1 中分割点为 i，在 nums2 中分割点为 j = (m+n+1)//2 - i

代码

```python
class Solution:
    def findMedianSortedArrays(self, nums1: List[int], nums2: List[int]) -> float:
        # 确保 nums1 是较短的数组
        if len(nums1) > len(nums2):
```

```
            nums1, nums2 = nums2, nums1

        m, n = len(nums1), len(nums2)
        # half 是左半部分应该有的元素个数
        half = (m + n + 1) // 2

        left, right = 0, m

        while left <= right:
            # i: nums1 左半部分的元素个数
            # j: nums2 左半部分的元素个数
            i = (left + right) // 2
            j = half - i

            # 获取分割线两侧的四个值
            nums1_left = float('-inf') if i == 0 else nums1[i - 1]
            nums1_right = float('inf') if i == m else nums1[i]
            nums2_left = float('-inf') if j == 0 else nums2[j - 1]
            nums2_right = float('inf') if j == n else nums2[j]

            if nums1_left <= nums2_right and nums2_left <= nums1_right:
                # 找到正确的分割点
                if (m + n) % 2 == 1:
                    # 奇数：中位数是左半部分的最大值
                    return max(nums1_left, nums2_left)
                else:
                    # 偶数：中位数是左半部分最大值和右半部分最小值的平均
                    return (max(nums1_left, nums2_left) + min(nums1_right, nums2_right)) /
2
            elif nums1_left > nums2_right:
                # nums1 左边太大，i 需要左移
                right = i - 1
            else:
                # nums2 左边太大，i 需要右移
                left = i + 1

        return 0.0
```

二分查找总结

题目	核心技巧
35. 搜索插入位置	标准二分模板，返回 left
74. 搜索二维矩阵	二维转一维：idx -> (idx//n, idx%n)
34. 查找第一个和最后一个位置	两次二分，分别找左右边界
33. 搜索旋转排序数组	判断哪半部分有序，再决定搜索方向
153. 寻找旋转排序数组最小值	与 nums[right] 比较确定搜索方向

4. 两个正序数组的中位数	在短数组上二分，找正确分割点

LeetCode Hot100 - 栈专题

---

20. 有效的括号 【简单】

思路

使用栈匹配括号：遇到左括号入栈，遇到右括号检查栈顶是否匹配。

- 右括号必须与最近的左括号配对 -> 后进先出 -> 栈
- 用哈希表存储括号对应关系，右括号作为 key

代码

```
class Solution:
    def isValid(self, s: str) -> bool:
        stack = []
        # 右括号 -> 左括号 的映射
        mapping = {')': '(', ']': '[', '}': '{'}

        for char in s:
            if char in mapping:
                # 遇到右括号，检查栈顶是否匹配
                if not stack or stack[-1] != mapping[char]:
                    return False
                stack.pop()
            else:
                # 左括号入栈
                stack.append(char)

        # 栈空则全部匹配成功
        return len(stack) == 0
```

---

155. 最小栈 【中等】

思路

使用辅助栈同步存储当前最小值：

- 主栈正常存数据
- 辅助栈栈顶始终保存当前主栈中的最小值
- 每次 push 时，辅助栈压入 min(当前值, 辅助栈顶)

代码

```
class MinStack:
    def __init__(self):
        self.stack = []        # 主栈
        self.min_stack = []    # 辅助栈，存储对应位置的最小值
```

```python
    def push(self, val: int) -> None:
        self.stack.append(val)
        # 辅助栈压入当前最小值
        if not self.min_stack:
            self.min_stack.append(val)
        else:
            self.min_stack.append(min(val, self.min_stack[-1]))

    def pop(self) -> None:
        self.stack.pop()
        self.min_stack.pop()  # 同步弹出

    def top(self) -> int:
        return self.stack[-1]

    def getMin(self) -> int:
        return self.min_stack[-1]  # O(1) 获取最小值
```

---

394. 字符串解码 【中等】

思路

双栈法：数字栈 + 字符串栈

- 遇到数字：累积数字
- 遇到 `[`：当前数字和字符串入栈，重置
- 遇到 `]`：弹出数字和之前的字符串，拼接 = 之前字符串 + 当前字符串 * 数字
- 遇到字母：追加到当前字符串

代码

```python
class Solution:
    def decodeString(self, s: str) -> str:
        num_stack = []    # 存储重复次数
        str_stack = []    # 存储之前的字符串
        cur_str = ""      # 当前正在构建的字符串
        cur_num = 0       # 当前数字

        for char in s:
            if char.isdigit():
                # 数字可能多位，累积计算
                cur_num = cur_num * 10 + int(char)
            elif char == '[':
                # 保存当前状态，开始新的一层
                num_stack.append(cur_num)
                str_stack.append(cur_str)
                cur_str = ""
                cur_num = 0
            elif char == ']':
```

```
                # 弹出并拼接：之前的字符串 + 当前字符串重复n次
                repeat = num_stack.pop()
                prev_str = str_stack.pop()
                cur_str = prev_str + cur_str * repeat
            else:
                # 字母直接追加
                cur_str += char

        return cur_str
```

---

739. 每日温度 【中等】

思路

单调递减栈（存下标）：

- 栈中保持温度递减，遇到更高温度时，弹出所有比它低的
- 弹出时计算等待天数 = 当前下标 - 弹出元素下标
- 存下标而非值，方便计算天数差

代码

```python
class Solution:
    def dailyTemperatures(self, temperatures: List[int]) -> List[int]:
        n = len(temperatures)
        answer = [0] * n
        stack = []  # 单调递减栈，存储下标

        for i, temp in enumerate(temperatures):
            # 当前温度比栈顶高，说明找到了栈顶元素的"下一个更高温度"
            while stack and temp > temperatures[stack[-1]]:
                prev_idx = stack.pop()
                answer[prev_idx] = i - prev_idx  # 等待天数
            stack.append(i)

        # 栈中剩余元素没有更高温度，保持默认值0
        return answer
```

---

84. 柱状图中最大的矩形 【困难】

思路

单调递增栈 + 哨兵：

- 对每个柱子，找左右两边第一个比它矮的位置，计算以它为高的最大矩形
- 单调递增栈：遇到更矮的柱子时，弹出栈顶计算面积
- 首尾添加高度为0的哨兵，简化边界处理
- 面积 = 高度 * (右边界 - 左边界 - 1)

代码

```python
class Solution:
    def largestRectangleArea(self, heights: List[int]) -> int:
        # 添加哨兵：首尾各加一个0，避免边界判断
        heights = [0] + heights + [0]
        stack = []  # 单调递增栈，存储下标
        max_area = 0

        for i, h in enumerate(heights):
            # 当前高度小于栈顶，弹出计算面积
            while stack and h < heights[stack[-1]]:
                cur_height = heights[stack.pop()]
                # 宽度 = 右边界i - 左边界stack[-1] - 1
                width = i - stack[-1] - 1
                max_area = max(max_area, cur_height * width)
            stack.append(i)

        return max_area
```

总结

题目	核心技巧	关键点
20. 有效的括号	栈匹配	右括号作key的哈希表
155. 最小栈	辅助栈	同步维护最小值
394. 字符串解码	双栈	遇到[保存状态，遇到]恢复拼接
739. 每日温度	单调递减栈	存下标，计算距离
84. 柱状图最大矩形	单调递增栈+哨兵	弹出时计算面积

栈的核心应用场景：

1. **括号匹配**：后进先出的特性
2. **单调栈**：寻找下一个更大/更小元素
3. **表达式解析**：保存中间状态，遇到特定符号时处理

LeetCode Hot100 堆专题速记

215. 数组中的第K个最大元素 【中等】

思路

核心思想：小顶堆维护前K大元素

1. 维护一个大小为 K 的小顶堆
2. 遍历数组，堆满后只有比堆顶大的元素才能入堆（先弹出堆顶再入堆）
3. 最终堆顶就是第 K 大元素

为什么用小顶堆？ 小顶堆堆顶是最小值，堆中始终保持最大的 K 个元素，堆顶即为第 K 大。

时间复杂度： O(n log k)，每个元素最多一次入堆操作

代码

```python
import heapq
from typing import List

class Solution:
    def findKthLargest(self, nums: List[int], k: int) -> int:
        # 小顶堆，维护前k大的元素
        heap = []

        for num in nums:
            if len(heap) < k:
                # 堆未满，直接入堆
                heapq.heappush(heap, num)
            elif num > heap[0]:
                # 比堆顶大，替换堆顶
                heapq.heapreplace(heap, num)

        # 堆顶即为第k大元素
        return heap[0]
```

---

347. 前 K 个高频元素 【中等】

思路

核心思想：小顶堆维护前K高频元素

1. 先用哈希表统计每个元素的出现频率
2. 维护一个大小为 K 的小顶堆，按频率排序
3. 堆中始终保持频率最高的 K 个元素

关键点： Python 的 heapq 是小顶堆，堆中存储 (频率, 元素) 元组，自动按频率排序

时间复杂度： O(n log k)

代码

```python
import heapq
from typing import List
from collections import Counter

class Solution:
    def topKFrequent(self, nums: List[int], k: int) -> List[int]:
        # 统计频率
        count = Counter(nums)

        # 小顶堆维护前k高频元素，存储（频率，元素）
        heap = []

        for num, freq in count.items():
            if len(heap) < k:
```

```
            heapq.heappush(heap, (freq, num))
        elif freq > heap[0][0]:
            # 频率更高，替换堆顶
            heapq.heapreplace(heap, (freq, num))

    # 提取结果（只需要元素，不需要频率）
    return [item[1] for item in heap]
```

---

295. 数据流的中位数 【困难】

思路

核心思想：大顶堆 + 小顶堆，对半分

1. **大顶堆 max_heap**：存储较小的一半数字（取负实现大顶堆）
2. **小顶堆 min_heap**：存储较大的一半数字
3. **平衡规则**：max_heap 的元素数量 = min_heap 或 min_heap + 1

添加元素流程：

- 先加入 max_heap，再把 max_heap 的最大值移到 min_heap
- 若 min_heap 元素更多，把 min_heap 最小值移回 max_heap
- 保证 max_heap 元素数 >= min_heap 元素数

获取中位数：

- 奇数个：max_heap 堆顶
- 偶数个：两个堆顶的平均值

时间复杂度： 添加 O(log n)，获取中位数 O(1)

代码

```
import heapq

class MedianFinder:

    def __init__(self):
        # 大顶堆存较小一半（取负模拟）
        self.max_heap = []
        # 小顶堆存较大一半
        self.min_heap = []

    def addNum(self, num: int) -> None:
        # 先加入大顶堆（取负）
        heapq.heappush(self.max_heap, -num)

        # 把大顶堆最大值移到小顶堆
        heapq.heappush(self.min_heap, -heapq.heappop(self.max_heap))

        # 平衡：保证大顶堆元素数 >= 小顶堆
        if len(self.min_heap) > len(self.max_heap):
            heapq.heappush(self.max_heap, -heapq.heappop(self.min_heap))
```

```python
    def findMedian(self) -> float:
        if len(self.max_heap) > len(self.min_heap):
            # 奇数个，大顶堆堆顶
            return -self.max_heap[0]
        else:
            # 偶数个，两堆顶平均值
            return (-self.max_heap[0] + self.min_heap[0]) / 2
```

堆专题总结

题目	核心技巧	堆类型
215. 第K大元素	小顶堆维护前K大	小顶堆
347. 前K高频	统计频率 + 小顶堆维护前K高频	小顶堆
295. 数据流中位数	双堆对半分，大顶堆+小顶堆	大顶堆+小顶堆

记忆口诀：

- 求第 K 大/前 K 大 → 小顶堆（堆顶最小，保留最大的 K 个）
- 求第 K 小/前 K 小 → 大顶堆（堆顶最大，保留最小的 K 个）
- Python 大顶堆技巧：存负数

LeetCode Hot100 贪心专题

---

121. 买卖股票的最佳时机 【简单】

思路

核心思想：维护历史最低价，计算每天卖出的最大利润

- 遍历价格数组，维护到当前为止的最低买入价格
- 每天计算：如果今天卖出，利润 = 今天价格 - 历史最低价
- 更新最大利润

贪心策略：总是在最低点买入，在之后的每一天尝试卖出

代码

```python
class Solution:
    def maxProfit(self, prices: List[int]) -> int:
        min_price = float('inf')  # 历史最低价
        max_profit = 0  # 最大利润

        for price in prices:
            min_price = min(min_price, price)  # 更新最低买入价
            max_profit = max(max_profit, price - min_price)  # 更新最大利润

        return max_profit
```

---

55. 跳跃游戏 【中等】

思路

核心思想：维护能到达的最远位置

- 遍历数组，不断更新能跳到的最远位置
- 如果当前位置超过了能到达的最远位置，说明无法继续
- 如果最远位置 >= 最后一个下标，返回 True

贪心策略：每一步都尽可能跳得更远

代码

```
class Solution:
    def canJump(self, nums: List[int]) -> bool:
        max_reach = 0  # 能到达的最远位置
        n = len(nums)

        for i in range(n):
            if i > max_reach:  # 当前位置无法到达
                return False
            max_reach = max(max_reach, i + nums[i])  # 更新最远位置
            if max_reach >= n - 1:  # 已经能到达终点
                return True

        return True
```

---

45. 跳跃游戏 II 【中等】

思路

核心思想：每次跳跃覆盖的范围内，选择能跳得最远的位置

- 维护当前跳跃的边界 `end` 和能到达的最远位置 `max_reach`
- 遍历到边界时，必须跳一次，更新边界为最远位置
- 统计跳跃次数

贪心策略：在每一步可选范围内，选择下一步能跳得最远的

代码

```
class Solution:
    def jump(self, nums: List[int]) -> int:
        n = len(nums)
        if n == 1:
            return 0

        jumps = 0  # 跳跃次数
        end = 0  # 当前跳跃的边界
        max_reach = 0  # 能到达的最远位置
```

```
        for i in range(n - 1):  # 不需要遍历最后一个位置
            max_reach = max(max_reach, i + nums[i])  # 更新最远位置
            if i == end:  # 到达当前跳跃边界，必须跳
                jumps += 1
                end = max_reach  # 更新边界
                if end >= n - 1:  # 已能到达终点
                    break

        return jumps
```

---

763. 划分字母区间 【中等】

思路

核心思想：每个字母的最后出现位置决定了区间的最小范围

1. 预处理：记录每个字母最后出现的位置
2. 遍历字符串，维护当前区间的最远边界
3. 当遍历到区间边界时，说明区间内所有字母都不会在后面出现，可以划分

贪心策略：尽量让每个区间包含其所有字母的完整出现范围

代码

```
class Solution:
    def partitionLabels(self, s: str) -> List[int]:
        # 记录每个字母最后出现的位置
        last = {c: i for i, c in enumerate(s)}

        result = []
        start = 0  # 当前区间起点
        end = 0  # 当前区间终点

        for i, c in enumerate(s):
            end = max(end, last[c])  # 扩展区间到包含当前字母的最后位置
            if i == end:  # 到达区间终点，可以划分
                result.append(end - start + 1)
                start = i + 1  # 开始新区间

        return result
```

贪心算法总结

题目	贪心策略	关键变量
121 买卖股票	最低价买入，每天尝试卖出	min_price, max_profit
55 跳跃游戏	每步尽量跳最远	max_reach
45 跳跃游戏II	每次跳跃选最优落点	max_reach, end, jumps

763 划分字母区间	区间必须包含所有字母	last[], start, end

LeetCode Hot100 动态规划专题速记

---

70. 爬楼梯 【简单】

【思路】

- 经典斐波那契问题：到达第n阶的方法 = 到达第n-1阶的方法 + 到达第n-2阶的方法
- 状态转移：`dp[i] = dp[i-1] + dp[i-2]`
- 空间优化：只需保存前两个状态

【代码】

```
class Solution:
    def climbStairs(self, n: int) -> int:
        if n <= 2:
            return n
        # 空间优化：只保存前两个状态
        prev, curr = 1, 2
        for i in range(3, n + 1):
            prev, curr = curr, prev + curr
        return curr
```

---

118. 杨辉三角 【简单】

【思路】

- 每个位置的值 = 上一行左上角 + 上一行正上方
- 边界：每行首尾都是1
- 状态转移：`dp[i][j] = dp[i-1][j-1] + dp[i-1][j]`

【代码】

```
class Solution:
    def generate(self, numRows: int) -> List[List[int]]:
        result = []
        for i in range(numRows):
            row = [1] * (i + 1)  # 初始化当前行，全为1
            # 填充中间元素
            for j in range(1, i):
                row[j] = result[i-1][j-1] + result[i-1][j]
            result.append(row)
        return result
```

---

198. 打家劫舍 【中等】

【思路】

- 不能偷相邻房屋，对于第i间房屋：偷或不偷
- 状态转移：`dp[i] = max(dp[i-1], dp[i-2] + nums[i])`
- dp[i]表示偷到第i间房屋时的最大金额

【代码】

```
class Solution:
    def rob(self, nums: List[int]) -> int:
        if len(nums) <= 2:
            return max(nums)
        # 空间优化
        prev2, prev1 = nums[0], max(nums[0], nums[1])
        for i in range(2, len(nums)):
            prev2, prev1 = prev1, max(prev1, prev2 + nums[i])
        return prev1
```

---

279. 完全平方数 【中等】

【思路】

- 完全背包问题：物品是完全平方数1,4,9,16...，求凑成n的最少物品数
- 状态转移：`dp[i] = min(dp[i], dp[i - j*j] + 1)`，其中j*j <= i
- dp[i]表示凑成i所需的最少完全平方数个数

【代码】

```
class Solution:
    def numSquares(self, n: int) -> int:
        dp = [float('inf')] * (n + 1)
        dp[0] = 0  # 凑成0需要0个数
        for i in range(1, n + 1):
            j = 1
            while j * j <= i:
                dp[i] = min(dp[i], dp[i - j * j] + 1)
                j += 1
        return dp[n]
```

---

322. 零钱兑换 【中等】

【思路】

- 完全背包问题：每种硬币可用无限次，求凑成amount的最少硬币数
- 状态转移：`dp[i] = min(dp[i], dp[i - coin] + 1)`
- dp[i]表示凑成金额i所需的最少硬币数

【代码】

```
class Solution:
    def coinChange(self, coins: List[int], amount: int) -> int:
        dp = [float('inf')] * (amount + 1)
        dp[0] = 0  # 凑成0元需要0枚硬币
```

```
        for i in range(1, amount + 1):
            for coin in coins:
                if coin <= i and dp[i - coin] != float('inf'):
                    dp[i] = min(dp[i], dp[i - coin] + 1)
        return dp[amount] if dp[amount] != float('inf') else -1
```

---

139. 单词拆分 【中等】

【思路】

- dp[i]表示s[0:i]是否可以被拆分成字典中的单词
- 遍历所有可能的分割点j，检查s[0:j]可拆分且s[j:i]在字典中
- 状态转移：`dp[i] = dp[j] and s[j:i] in wordSet`

【代码】

```
class Solution:
    def wordBreak(self, s: str, wordDict: List[str]) -> bool:
        wordSet = set(wordDict)
        n = len(s)
        dp = [False] * (n + 1)
        dp[0] = True  # 空字符串可拆分

        for i in range(1, n + 1):
            for j in range(i):
                # s[0:j]可拆分 且 s[j:i]是字典中的单词
                if dp[j] and s[j:i] in wordSet:
                    dp[i] = True
                    break
        return dp[n]
```

---

300. 最长递增子序列 【中等】

【思路】

- dp[i]表示以nums[i]结尾的最长递增子序列长度
- 对于每个i，遍历所有j<i，若nums[j]<nums[i]则可接上
- 状态转移：`dp[i] = max(dp[j] + 1)`，其中j<i且nums[j]<nums[i]
- 优化：二分查找可达O(nlogn)

【代码】

```
class Solution:
    def lengthOfLIS(self, nums: List[int]) -> int:
        n = len(nums)
        dp = [1] * n  # 每个元素自身构成长度为1的子序列

        for i in range(1, n):
            for j in range(i):
                if nums[j] < nums[i]:
                    dp[i] = max(dp[i], dp[j] + 1)
```

```
        return max(dp)
```

【二分优化版本】

```
class Solution:
    def lengthOfLIS(self, nums: List[int]) -> int:
        # tails[i]表示长度为i+1的递增子序列的最小末尾元素
        tails = []
        for num in nums:
            # 二分查找num应该插入的位置
            left, right = 0, len(tails)
            while left < right:
                mid = (left + right) // 2
                if tails[mid] < num:
                    left = mid + 1
                else:
                    right = mid
            if left == len(tails):
                tails.append(num)
            else:
                tails[left] = num
        return len(tails)
```

---

152. 乘积最大子数组 【中等】

【思路】

- 因为负数乘负数会变大，需要同时维护最大值和最小值
- 状态转移：
  - `maxProd[i] = max(nums[i], maxProd[i-1]*nums[i], minProd[i-1]*nums[i])`
  - `minProd[i] = min(nums[i], maxProd[i-1]*nums[i], minProd[i-1]*nums[i])`

【代码】

```
class Solution:
    def maxProduct(self, nums: List[int]) -> int:
        n = len(nums)
        maxProd = minProd = result = nums[0]

        for i in range(1, n):
            # 注意：需要先保存maxProd，因为计算minProd时会用到原值
            temp = maxProd
            maxProd = max(nums[i], maxProd * nums[i], minProd * nums[i])
            minProd = min(nums[i], temp * nums[i], minProd * nums[i])
            result = max(result, maxProd)

        return result
```

---

416. 分割等和子集 【中等】

【思路】

- 转化为0-1背包：能否从数组中选出若干数，使其和等于总和的一半
- 总和为奇数直接返回False
- dp[j]表示是否能凑出和为j
- 状态转移：`dp[j] = dp[j] or dp[j - nums[i]]`

【代码】

```
class Solution:
    def canPartition(self, nums: List[int]) -> bool:
        total = sum(nums)
        if total % 2 == 1:  # 奇数无法平分
            return False

        target = total // 2
        dp = [False] * (target + 1)
        dp[0] = True  # 和为0总是可以达到

        for num in nums:
            # 倒序遍历，确保每个数只用一次（0-1背包）
            for j in range(target, num - 1, -1):
                dp[j] = dp[j] or dp[j - num]

        return dp[target]
```

---

32. 最长有效括号 【困难】

【思路】

- dp[i]表示以s[i]结尾的最长有效括号长度
- 只有s[i]==')'时才可能有效
- 两种情况：
  - s[i-1]=='('：`dp[i] = dp[i-2] + 2`
  - s[i-1]==')'：检查s[i-dp[i-1]-1]是否为'('

【代码】

```
class Solution:
    def longestValidParentheses(self, s: str) -> int:
        n = len(s)
        if n == 0:
            return 0

        dp = [0] * n
        maxLen = 0

        for i in range(1, n):
            if s[i] == ')':
```

```
            if s[i-1] == '(':
                # 形如 ...()
                dp[i] = (dp[i-2] if i >= 2 else 0) + 2
            elif i - dp[i-1] - 1 >= 0 and s[i - dp[i-1] - 1] == '(':
                # 形如 ...)), 检查与内部匹配的外部左括号
                dp[i] = dp[i-1] + 2
                # 加上外部左括号之前的有效长度
                if i - dp[i-1] - 2 >= 0:
                    dp[i] += dp[i - dp[i-1] - 2]
            maxLen = max(maxLen, dp[i])

        return maxLen
```

---

62. 不同路径 【中等】

【思路】

- 只能向右或向下走，到达(i,j)的路径数 = 从上方来 + 从左方来
- 状态转移：`dp[i][j] = dp[i-1][j] + dp[i][j-1]`
- 边界：第一行和第一列都只有1条路径

【代码】

```
class Solution:
    def uniquePaths(self, m: int, n: int) -> int:
        # 空间优化：只用一维数组
        dp = [1] * n  # 第一行全为1

        for i in range(1, m):
            for j in range(1, n):
                dp[j] = dp[j] + dp[j-1]  # 上方 + 左方

        return dp[n-1]
```

---

64. 最小路径和 【中等】

【思路】

- 到达(i,j)的最小路径和 = min(从上方来, 从左方来) + 当前格子值
- 状态转移：`dp[i][j] = min(dp[i-1][j], dp[i][j-1]) + grid[i][j]`
- 边界：第一行只能从左来，第一列只能从上来

【代码】

```
class Solution:
    def minPathSum(self, grid: List[List[int]]) -> int:
        m, n = len(grid), len(grid[0])
        # 空间优化：使用一维数组
        dp = [0] * n

        for i in range(m):
```

```
            for j in range(n):
                if i == 0 and j == 0:
                    dp[j] = grid[0][0]
                elif i == 0:
                    dp[j] = dp[j-1] + grid[i][j]  # 第一行只能从左来
                elif j == 0:
                    dp[j] = dp[j] + grid[i][j]    # 第一列只能从上来
                else:
                    dp[j] = min(dp[j], dp[j-1]) + grid[i][j]

        return dp[n-1]
```

---

5. 最长回文子串 【中等】

【思路】

- 方法1：DP，dp[i][j]表示s[i:j+1]是否为回文
- 方法2：中心扩展法，枚举每个中心点向两边扩展
- 状态转移：`dp[i][j] = (s[i] == s[j]) and dp[i+1][j-1]`

【代码】

```
class Solution:
    def longestPalindrome(self, s: str) -> str:
        n = len(s)
        if n < 2:
            return s

        start, maxLen = 0, 1

        def expandAroundCenter(left: int, right: int) -> int:
            """从中心向两边扩展，返回回文长度"""
            while left >= 0 and right < n and s[left] == s[right]:
                left -= 1
                right += 1
            return right - left - 1

        for i in range(n):
            len1 = expandAroundCenter(i, i)      # 奇数长度回文
            len2 = expandAroundCenter(i, i + 1)  # 偶数长度回文
            curLen = max(len1, len2)
            if curLen > maxLen:
                maxLen = curLen
                start = i - (curLen - 1) // 2

        return s[start:start + maxLen]
```

1143. 最长公共子序列 【中等】

【思路】

- dp[i][j]表示text1[0:i]和text2[0:j]的最长公共子序列长度
- 若text1[i-1] == text2[j-1]：`dp[i][j] = dp[i-1][j-1] + 1`
- 否则：`dp[i][j] = max(dp[i-1][j], dp[i][j-1])`

【代码】

```
class Solution:
    def longestCommonSubsequence(self, text1: str, text2: str) -> int:
        m, n = len(text1), len(text2)
        # dp[i][j]表示text1前i个字符和text2前j个字符的LCS长度
        dp = [[0] * (n + 1) for _ in range(m + 1)]

        for i in range(1, m + 1):
            for j in range(1, n + 1):
                if text1[i-1] == text2[j-1]:
                    dp[i][j] = dp[i-1][j-1] + 1
                else:
                    dp[i][j] = max(dp[i-1][j], dp[i][j-1])

        return dp[m][n]
```

---

72. 编辑距离 【中等】

【思路】

- dp[i][j]表示word1[0:i]转换为word2[0:j]所需的最少操作数
- 三种操作：插入、删除、替换
- 状态转移：
  - 若word1[i-1] == word2[j-1]：`dp[i][j] = dp[i-1][j-1]`
  - 否则：`dp[i][j] = min(dp[i-1][j], dp[i][j-1], dp[i-1][j-1]) + 1`
    - dp[i-1][j] + 1：删除word1[i-1]
    - dp[i][j-1] + 1：插入word2[j-1]
    - dp[i-1][j-1] + 1：替换

【代码】

```
class Solution:
    def minDistance(self, word1: str, word2: str) -> int:
        m, n = len(word1), len(word2)
        # dp[i][j]表示word1前i个字符转换为word2前j个字符的最少操作数
        dp = [[0] * (n + 1) for _ in range(m + 1)]

        # 初始化边界
        for i in range(m + 1):
            dp[i][0] = i  # word1删除i个字符变成空串
        for j in range(n + 1):
            dp[0][j] = j  # 空串插入j个字符变成word2

        for i in range(1, m + 1):
            for j in range(1, n + 1):
```

```
            if word1[i-1] == word2[j-1]:
                dp[i][j] = dp[i-1][j-1]  # 字符相同，无需操作
            else:
                dp[i][j] = min(
                    dp[i-1][j] + 1,    # 删除
                    dp[i][j-1] + 1,    # 插入
                    dp[i-1][j-1] + 1   # 替换
                )

    return dp[m][n]
```

动态规划解题模板总结

核心步骤

1. **定义状态**：明确dp数组的含义
2. **初始化**：确定边界条件
3. **状态转移**：找出递推关系
4. **遍历顺序**：根据依赖关系确定遍历方向
5. **返回结果**：通常是dp数组的最后一个元素或最大/最小值

常见类型

类型	典型题目	特点
线性DP	爬楼梯、打家劫舍	一维状态，依赖前几个状态
背包DP	零钱兑换、分割等和子集	物品+容量的选择问题
区间DP	最长回文子串、最长有效括号	区间合并或分割
二维DP	不同路径、最小路径和	网格型问题
序列DP	LIS、LCS、编辑距离	两个序列的匹配问题

空间优化技巧

- 一维优化：当前状态只依赖上一行时，可用一维数组
- 滚动数组：交替使用两行
- 变量优化：只依赖有限个状态时，用变量代替数组

LeetCode Hot100 技巧专题速记

---

136. 只出现一次的数字 【简单】

思路：

- 核心：异或运算的性质
- `a ^ a = 0`（相同数异或为0）
- `a ^ 0 = a`（任何数与0异或为自身）
- 将所有数异或，成对的数会抵消为0，最后剩下的就是只出现一次的数

代码：

```python
class Solution:
    def singleNumber(self, nums: List[int]) -> int:
        result = 0
        for num in nums:
            result ^= num  # 异或：相同为0，不同为1
        return result
```

---

169. 多数元素 【简单】

思路：

- 核心：Boyer-Moore 投票算法
- 维护一个候选人和票数计数器
- 遇到相同的数，票数+1；遇到不同的数，票数-1
- 票数为0时，更换候选人
- 多数元素（出现次数 > n/2）最终一定会胜出

代码：

```python
class Solution:
    def majorityElement(self, nums: List[int]) -> int:
        candidate = None
        count = 0

        for num in nums:
            if count == 0:
                candidate = num  # 票数归零，更换候选人
            count += 1 if num == candidate else -1  # 同则+1，异则-1

        return candidate
```

---

75. 颜色分类 【中等】

思路：

- 核心：荷兰国旗问题，三指针原地排序
- `p0` 指向0的右边界， `p2` 指向2的左边界
- `curr` 遍历数组：
  - 遇到0：与 `p0` 交换， `p0++` ， `curr++`
  - 遇到2：与 `p2` 交换， `p2--` （curr不动，因为交换来的数还需判断）
  - 遇到1： `curr++`

代码：

```python
class Solution:
    def sortColors(self, nums: List[int]) -> None:
        p0, curr, p2 = 0, 0, len(nums) - 1

        while curr <= p2:
```

```
        if nums[curr] == 0:
            nums[p0], nums[curr] = nums[curr], nums[p0]
            p0 += 1
            curr += 1  # p0左边都是0，交换来的一定是1，可以前进
        elif nums[curr] == 2:
            nums[curr], nums[p2] = nums[p2], nums[curr]
            p2 -= 1    # 交换来的数未知，curr不动继续判断
        else:
            curr += 1  # 是1，直接跳过
```

---

31. 下一个排列 【中等】

思路：

- 目标：找到下一个字典序更大的排列
- 步骤：
  1. 从右往左找第一个下降的位置 `i` （`nums[i] < nums[i+1]`）
  2. 从右往左找第一个大于 `nums[i]` 的数，与 `nums[i]` 交换
  3. 将 `i` 后面的部分反转（从降序变升序，得到最小排列）
- 若找不到下降位置，说明是最大排列，直接整体反转

代码：

```
class Solution:
    def nextPermutation(self, nums: List[int]) -> None:
        n = len(nums)
        i = n - 2

        # 1. 从右往左找第一个下降点
        while i >= 0 and nums[i] >= nums[i + 1]:
            i -= 1

        if i >= 0:
            # 2. 从右往左找第一个大于nums[i]的数并交换
            j = n - 1
            while nums[j] <= nums[i]:
                j -= 1
            nums[i], nums[j] = nums[j], nums[i]

        # 3. 反转i后面的部分
        left, right = i + 1, n - 1
        while left < right:
            nums[left], nums[right] = nums[right], nums[left]
            left += 1
            right -= 1
```

---

287. 寻找重复数 【中等】

思路：

- 核心：快慢指针（Floyd 判圈算法）
- 把数组看成链表：`index -> nums[index]`
- 有重复数意味着有多个位置指向同一个点，形成环
- 步骤：
  1. 快慢指针找相遇点（确认有环）
  2. 一个指针回到起点，两指针同速前进，再次相遇点就是环入口（重复数）

代码：

```
class Solution:
    def findDuplicate(self, nums: List[int]) -> int:
        # 1. 快慢指针找相遇点
        slow = fast = 0
        while True:
            slow = nums[slow]          # 走一步
            fast = nums[nums[fast]]    # 走两步
            if slow == fast:
                break

        # 2. 找环入口（重复数）
        slow = 0
        while slow != fast:
            slow = nums[slow]
            fast = nums[fast]

        return slow
```

技巧总结

题目	核心技巧	时间复杂度	空间复杂度
136. 只出现一次的数字	异或运算	O(n)	O(1)
169. 多数元素	Boyer-Moore投票	O(n)	O(1)
75. 颜色分类	荷兰国旗/三指针	O(n)	O(1)
31. 下一个排列	找规律+双指针	O(n)	O(1)
287. 寻找重复数	Floyd判圈算法	O(n)	O(1)